Industrial Internship Report on

"Quiz Game Application"

Prepared by

Said Mallick

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. I had to finish the project including the report in 4 weeks' time. My project was the development of a Quiz Game Application using Python. Over the four weeks, the project evolved from a basic console-based script to a fully functional desktop application with a Graphical User Interface (GUI), an SQLite database, and multi-user support. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement a solution for them. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 3

2 Introduction....5

2.1 About UniConverge Technologies Pvt Ltd....5

2.2 About upskill Campus....9

2.3 Objective 11

2.4 Reference 11

2.5 Glossary....11

3 Problem Statement....11

4 Existing and Proposed solution 12

5 Proposed Design/ Model....13

5.1 High Level Diagram (if applicable) 13

5.2 Low Level Diagram (if applicable)....13

5.3 Interfaces (if applicable)....13

6 Performance Test 14

6.1 Test Plan/ Test Cases 14

6.2 Test Procedure....15

6.3 Performance Outcome....15

7 My learnings....15

8 Future work scope 16

1 Preface

Summary of the whole 6 weeks' work:

During the first week, I built the core logic of the Quiz Game using Python, storing questions in a JSON file. In the second week, I added topic categories (Science, History, etc.) and a score-tracking system. The third week focused on transforming the console app into a desktop application using Tkinter, adding a countdown timer and difficulty levels. The final week involved migrating the data to an SQLite database, adding user profiles, and building an Admin Panel to manage questions.

About need of relevant Internship in career development:

An internship bridges the gap between academic theory and real-world application. It teaches practical skills like database management, GUI design, and debugging, which are essential for a career in software development.

Brief about Your project/problem statement:

The project is a desktop-based Quiz Game that allows multiple users to log in, select a topic and difficulty level, answer timed multiple-choice questions, and save their scores to a database. It also includes an admin panel for managing the question bank.

Opportunity given by USC/UCT:

I am grateful to USC and UCT for providing this structured internship program, which allowed me to work on a real-world project and receive continuous mentorship.

How Program was planned:

The program was planned in a step-by-step manner over 6 weeks:

1. Explore the problem statement and set up the environment.
2. Implement core logic and data storage.
3. Develop the user interface and interactive features.
4. Integrate the database, validate performance, and finalize the submission.

Your Learnings and overall experience:

I learned how to structure a complete Python application, manage a relational database, and handle user input through a GUI. The overall experience was highly enriching and boosted my confidence as a developer.

Thank to all (with names), who have helped you directly or indirectly:

I would like to thank my mentors, the USC/UCT coordinators, and my peers for their constant guidance and support throughout this internship.

Your message to your juniors and peers:

I would encourage my juniors to focus on building projects from scratch. Break down complex problems into smaller tasks, and don't be afraid to make mistakes—debugging is the best way to learn.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



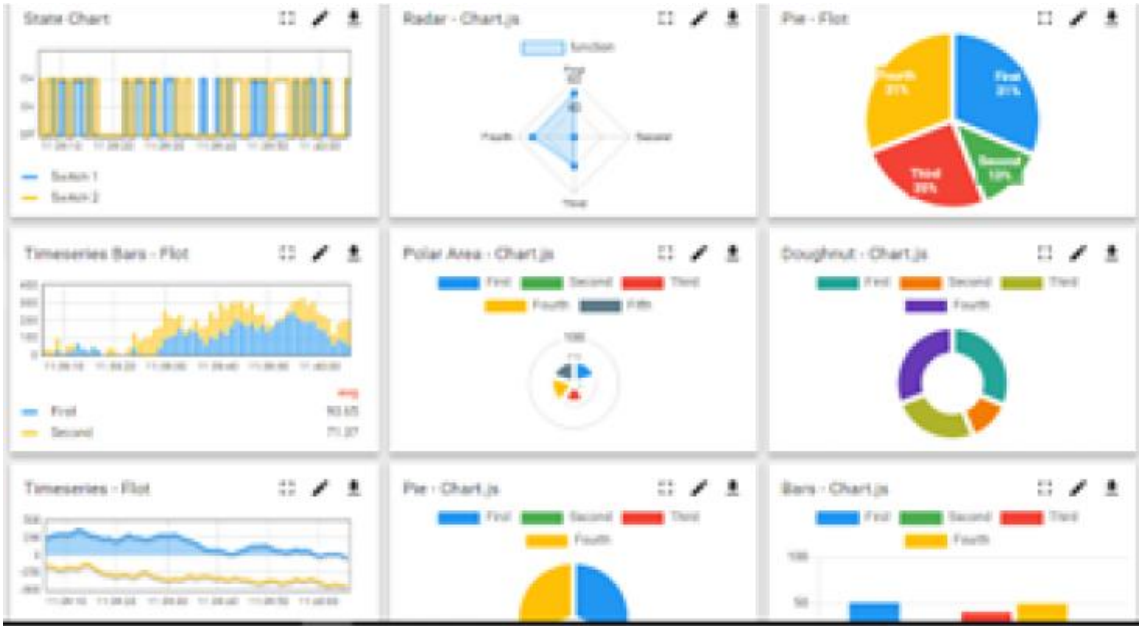
i. UCT IoT Platform (UCT Insight)

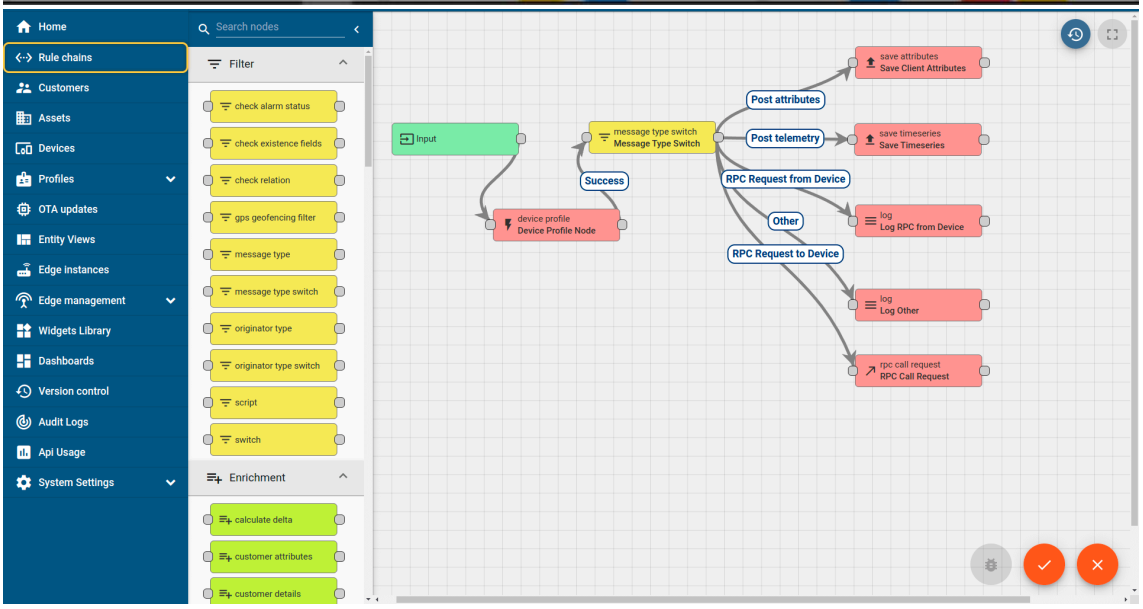
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

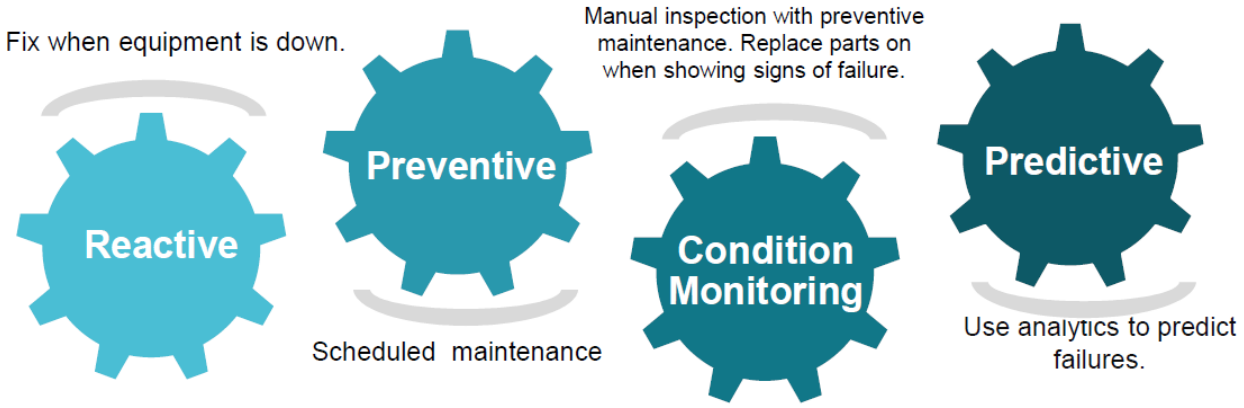


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

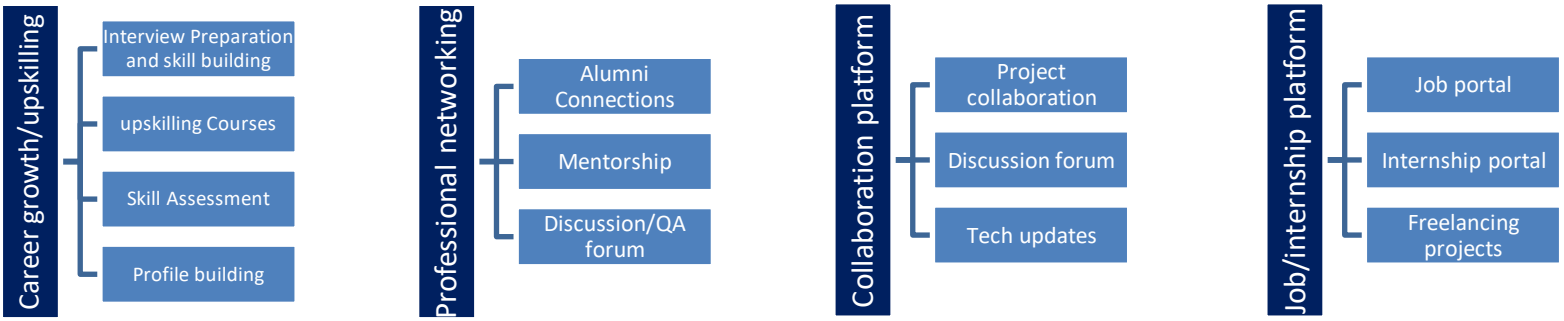
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to:

- Get practical experience of working in the industry.
- Solve real-world problems.
- Have improved job prospects.
- Have an improved understanding of our field and its applications.
- Have personal growth like better communication and problem-solving.

2.5 Reference

[1] Python Official Documentation – docs.python.org

[2] Tkinter Documentation – docs.python.org/3/library/tkinter.html

[3] SQLite Documentation – sqlite.org/docs.html

2.6 Glossary

Terms	Acronym
GUI	Graphical user interface
SQLite	A lightweight, file-based relational database
JSON	JavaScript object notation (used for initial data seeding)
Tkinter	Python's standard GUI library
CRUD	Create, Read, Update, Delete
DB	Database

3 Problem Statement

Traditional quizzes are often static, paper-based, or require an internet connection to access online platforms. There is a need for an offline, desktop-based application that allows multiple users to test their knowledge across various topics, track their progress over time, and allow administrators to easily manage the question bank without needing to edit code or text files manually.

4 Existing and Proposed solution

Existing Solutions and Limitations:

Existing solutions include online platforms like Kahoot or Quizizz, which require a stable internet connection and often have subscription fees. Basic Python quiz scripts exist, but they are console-based, lack a GUI, do not store persistent scores, and are limited to a single user.

Proposed Solution:

I proposed and built a standalone Python desktop application using Tkinter for the GUI and SQLite for the database. The application offers:

- A user-friendly graphical interface.
- Multiple topics and difficulty levels (Easy, Medium, Hard).
- A per-question countdown timer.
- Multi-user support with individual score histories.
- An Admin Panel to add or delete questions dynamically.

Value Addition:

The application works completely offline, stores data persistently, supports multiple users, and provides a seamless, interactive experience without requiring any technical knowledge from the end-user.

4.1 Code submission (Github link)

4.2 Report submission (Github link) :

5 Proposed Design/ Model

Design Flow:
The application follows a three-layer architecture:
Presentation Layer (UI): Built with Tkinter. Handles all user interactions, buttons, windows, and timers.
Logic Layer: The Quiz Engine. Handles question flow, answer validation, score calculation, and timer countdown logic.
Data Layer: SQLite Database. Stores questions and user scores. A JSON file is used only once to seed the initial questions.

5.1 High Level Diagram (if applicable)

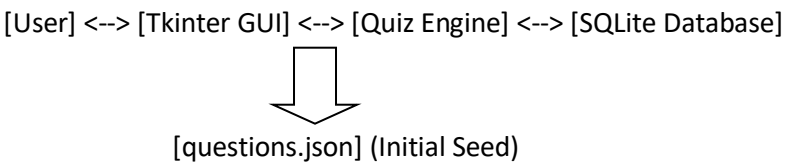


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

User Flow:

Login -> Main Menu -> Select Topic & Difficulty -> Loop through Questions (Display Question -> Start Timer -> Get Answer -> Check Answer) -> Save Score to DB -> Show Result Screen -> View History/Stats.

Admin Flow:

Admin Panel -> View Questions -> Add Question (Form) -> Save to DB.

Admin Panel -> Delete Question -> Remove from DB.

5.3 Interfaces (if applicable)

Main Menu Interface: Dropdowns for topic/difficulty, buttons for Start, History, Stats, Admin, and Exit.
Quiz Interface: Question label, 4 option buttons, timer label, and score label.
Result Interface: Final score, percentage, and motivational message.
Admin Interface: Listbox to display questions, buttons to Add/Delete/Refresh.

6 Performance Test

Constraints:

The main constraints identified were:

Responsiveness: The GUI must not freeze while the timer is running.

Data Integrity: Deleting a question should not corrupt existing score records.

Speed: Database queries must return results instantly.

How constraints were taken care of in the design:

Used Tkinter's after() method instead of time.sleep() to keep the GUI responsive during the countdown.

Scores were stored independently with topic/difficulty rather than question IDs, ensuring data integrity even if a question is deleted.

Parameterized SQL queries were used for fast and secure database operations.

6.1 Test Plan/ Test Cases

Test Case	Description	Expected Result
TC-01	Launch application	Login prompt appears
TC-02	Enter username and start quiz	Main menu loads with topics
TC-03	Select topic and difficulty	Quiz starts with correct filter
TC-04	Timer countdown	Timer decreases from 30s/20s/10s correctly
TC-05	Answer correctly	Score increments, popup shows "Correct"
TC-06	Answer incorrectly	Popup shows "Wrong" with correct answer
TC-07	Let timer run out	Popup shows "Time's Up", moves to next question
TC-08	Finish quiz	Score saved to DB, result screen shown
TC-09	View History	Previous scores appear in a popup window
TC-10	Admin - Add Question	New question appears in the list and DB
TC-11	Admin - Delete Question	Question is removed from the list and DB

6.2 Test Procedure

Manual testing was performed by running the application multiple times, simulating both valid and invalid user inputs (e.g., clicking answers rapidly, letting the timer expire, adding empty questions in the admin panel). The SQLite database was inspected using DB Browser for SQLite to verify data storage.

6.3 Performance Outcome

- The application launches within 1-2 seconds.
- The timer is accurate to the second and does not cause the GUI to freeze.
- Database read/write operations complete in less than 10 milliseconds.
- No crashes or memory leaks were observed during continuous testing.
- All 11 test cases passed successfully.

7 My learnings

Over the past 4 weeks, I gained immense practical knowledge. I learned how to:

- Write clean, modular Python code.
- Handle files using JSON and perform CRUD operations in SQLite.
- Build interactive desktop applications using Tkinter, including event handling and window management.
- Implement a countdown timer without freezing the GUI.
- Design a database schema and write secure SQL queries.
- Debug complex issues and structure a project for scalability.

These skills will directly help me in my career growth as a Python developer, as I now understand how to build a complete, database-driven application from scratch.

8 Future work scope

Due to time limitations, some features could not be implemented but can be taken up in the future:

- **Cloud Integration:** Sync scores to a cloud database so users can access their history from any device.
- **Multiplayer Mode:** Allow two users to compete in real-time over a local network.
- **AI-Generated Questions**: Use an API to generate dynamic questions based on a chosen topic.
- **Mobile App Port**: Convert the application into a mobile app using Kivy or BeeWare.
- **Advanced Analytics:** Add graphs and charts to show user progress over time.